Deep Reinforcement Learning-Based Mobile-Aware Edge Cooperative Caching Scheme in the Internet of Vehicles

Weidi Tian, Yujian Chen, Feng Ke, and Hui Song

***Abstract*—The number of computation-intensive apps and content requests within the Internet of Vehicles (IoV) has dramatically expanded with the swift advancement of 5G communication technologies and next-generation Internet technology. To address the issue of excessive response latency caused by the surge in data traffic, edge caching (EC) technology has proven to be effective. However, accurately predicting the popularity of content in different regions is challenging due to the high degree of mobility of the vehicles. Additionally, as vehicles typically do not remain in the same region for a long period of time, making the content popularity within a region highly dynamic. To address this issue, we propose a deep reinforcement learning-based mobile-aware edge cooperative caching scheme (MAECCD). Our approach utilizes an AutoEncoder to capture user features from local vehicle user data and accurately forecast content popularity in both vehicle-local and edge regions. Furthermore, the MAECCD incorporates a cooperation model between edge nodes to optimize cache utilization and employs techniques in Deep Reinforcement Learning (DRL) to refresh cached content and reduce transmission delay. Experimental results indicate that our MAECCD scheme outperforms other baseline algorithms in terms of cache hit ratio and content transmission delay.**

***Index Terms*—IoV, cooperative caching, deep reinforcement learning, federated learning.**

I. Introduction

THE rapid development of wireless communication technology and the Internet of Vehicles (IoV) have raised the bar for Intelligent Transportation Systems (ITS). With the widespread use of advanced applications such as autonomous driving technology, in-vehicle video, smart voice, and smart map navigation, the demand for content in Vehicular Networks (VNs) is experiencing explosive growth [1]. This surge in data traffic will have a significant influence on the network efficiency of VNs, including network capacity, network quality of service, and content transmission delay. It also places higher demands on wireless data services and mobile networks [2], [3]. While cloud servers and Macro Base Stations (MBS) can meet the diverse data content needs of vehicle users, providing a low-latency network service may not always be feasible. In addition, there is the considerable distance between the vehicles and the cloud servers, as well as the substantial data traffic, can lead to communication congestion and add pressure to backhaul links [4]. However, the emergence of Edge Computing (EC) technology offers the potential for low-latency and low-energy, VNs services [5].

By integrating EC technology into IoV environments, network functions such as computation, communication, and storage can be moved to roadway road test units (RSUs) at the edges of VNs [6]. This approach takes advantage of the physical proximity of RSUs to vehicles, reducing communication congestion and improving reliability, which can be negatively affected by long-distance transmission [7]. Specifically, the vehicle request content is pre-cached into the vehicle and nearby RSUs, and the user retrieves the necessary content from the nearby nodes via vehicle-to-vehicle (V2V) and vehicle-to-infrastructure (V2I) communication links. This reduces the need for backhaul links to obtain the content from the remote servers, thus decreasing the content acquisition latency and alleviating the burden of massive data traffic on the backhaul link [8]. However, the storage capacity of both vehicles and RSUs is limited, making it hard to pre-cache everything. As the number of requests increases, effectively utilizing the limited storage space to achieve maximum bandwidth savings, improve request hit rate, and reduce content transmission delay becomes an pressing issue that needs to be addressed [9]. This issue of inadequate cache capacity in edge nodes within IoV environments can be resolved through two means: 1) Precisely predicting content popularity to enhance the hit rate of edge caches. 2) Utilizing the ability of edge nodes to cooperate and improve cache usage on edge devices.

A significant proportion of data traffic in IoV environments comes from duplicated popular content [10], causing substantial wastage of network resources and impairing vehicle users' quality of experience (QoE). Therefore, accurately predicting content popularity is crucial when designing an IoV caching scheme. The traditional caching approach is passive and does

Received 13 September 2024; revised 12 November 2024; accepted 14 November 2024. Date of publication 18 November 2024; date of current version 5 March 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 62171188, in part by the National YouthFund of China under Grant 61801184, and in part by the Guangzhou City Science and Technology Plan Project under Grant 202002020019. The review of this article was coordinated by Prof. Daniele Tarchi. *(Corresponding author: Hui Song.)*

Weidi Tian, Yujian Chen, and Hui Song are with the School of Electronic Science and Engineering (School of Microelectronics), South China Normal University, Guangzhou 510631, China (e-mail: 2022024880@m.scnu.edu.cn; 2022024862@m.scnu.edu.cn; songhui@m.scnu.edu.cn).

Feng Ke is with the School of Electronic and Information Engineering, South China University of Technology, Guangzhou 510641, China (e-mail: fengke@scut.edu.cn).

Digital Object Identifier 10.1109/TVT.2024.3501364

0018-9545 © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: NORTHWESTERN POLYTECHNICAL UNIVERSITY. Downloaded on May 20,2026 at 02:37:52 UTC from IEEE Xplore. Restrictions apply.

not consider the impact of popular content [11]. Given the high mobility characteristics of moving vehicles, the active caching approach is more suitable for dynamically changing IoV environments. Machine Learning (ML), a branch of the Artificial Intelligence (AI), offers new ideas for predicting content popularity in active caching [12]. While ML is effective in predicting content popularity, it presents challenges in IoV environments. Traditional ML training methods are centralized and require collecting user data on a server before training. However, because of the sensitive nature of the information, centralized training is not suitable for predicting popularity. Federated Learning (FL) is a distributed learning framework that allows trainers to training locally and use model updates to communicate between vehicles and edge servers, effectively addressing privacy concerns [13], [14]. Another challenge in IoV edge active caching is the potential for quickly outdated predictions due to the high degree of mobility of the vehicles. Consequently, active caching schemes should account for the effect of the manoeuvrability of vehicles and dynamically adjust the cached content [15].

Another way to address the limited caching capabilities of edge devices is to utilize the cooperation capabilities among nodes [16]. While cooperation is possible between vehicles, between vehicles and RSUs, and between RSUs in IoV environments, existing studies do not fully utilize the cache resources between nodes [17], [18]. Thus, it is crucial to consider the cooperation capability among nodes when designing IoV edge caching schemes to reduce resource waste and improve the cache utilization on edge devices. Traditional approaches are not well-suited for dynamic IoV environments due to high vehicle mobility and the complexity of content delivery, making DRL-based approaches a more suitable option for IoV edge caching.

Several factors, such as the real-time popularity, the dynamic nature of content requests of content, the security of user data, the high mobility of vehicles, and limited computing power and cache capacity in the complex and dynamic environment of the IoV, can significantly affect the performance of edge caching schemes. In light of these challenges, this paper proposes a mobile-aware edge cooperative caching scheme (MAECCD) based on FL and DRL for the Internet of Vehicles. The main objective of this scheme is to minimize content transmission delay by considering various factors, including vehicle mobility.

The main contributions of this paper are summarized as follows:

1) We propose a method for predicting popularity that is based on AutoEncoder(AE) and FL, and can be adapted to the dynamic environment of IoV. The training update of the model adopts the asynchronous FL framework, which effectively reduces the training delay and protects the user's privacy. The effects of vehicle speed, location, transmission rate and cache capacity are considered to improve the accuracy of the model.
2) The high-order features extracted by AutoEncoder are used to predict the popular content. Method Collaborative filtering algorithm is used to obtain the correlation between users to predict the user's rating of unrated content, and the user's preference for different themes is predicted according to the user's historical request data.
3) We propose MAECCD, an edge cooperative caching scheme for the Internet of Vehicles. This scheme collects vehicle information at intersection RSUs and captures vehicle user profiles to predict content popularity for each vehicle and the region as a whole. Popular content is then cached at each caching node. As the vehicle moves to the next region, the predicted content popularity follows and utilizes the next RSU in the direction of travel of the vehicle as a cooperative cache node.
4) The MAECCD scheme utilises the Dueling DQN algorithm in DRL to update the content in both the vehicle and RSU cache, resulting in minimal content transmission delay. Before the algorithm is initiated, vehicles are efficiently clustered based on their speed, position, and V2V communication distance in order to reduce communication overhead.

The rest of this article is organized as follows. Section II provides an overview of the existing research on edge caching. In Section III, the system model is briefly explained. Section IV introduces a popularity prediction scheme utilizing asynchronous FL. Section V proposes a mobile-aware collaborative caching scheme based on DRL. Simulation results and performance analysis are presented in Section VI. Lastly, Section VII provides a summary of the paper.

II. Related Work

Modeling content popularity using the Zipf model is an inadequate predictor of user requests for vehicles in existing edge caching strategies for the IoV. The popularity of content obtained through this approach is typically fixed at a certain time and may not be applicable to dynamic IoV environments. To address this issue, researchers in [19] utilized neural networks to enhance the inputs to the Zipf model in their proposed cooperative caching approach for the IoV. They also investigated the covert temporal correlation laws from the users' historical request information and used a Long Short-Term Memory (LSTM) network to predict the number of requests for the vehicle's next time period, thereby obtaining more accurate content popularity. In [20], Yu et al. not only considered vehicle user history requests, but also incorporated user contextual information. They used cooperative filtering algorithms and a Variational Auto-Encoder (VAE) to predict document popularity. To further improve the accuracy of predictions and reduce the impact of wandering vehicles, they proposed a double-weighted model aggregation scheme. In contrast, the edge cache replacement scheme proposed in [21] utilized a context-aware Adversarial Auto-Encoder to predict content popularity while also using FL to protect user privacy. [22] The proposed edge caching scheme for the IoV employed an AutoEncoder to reconstruct historical user rating information. This information was then used to obtain user ratings for unrated content and predict popular content using the reconstructed ratings. As a result, the content delivery delay in the caching scheme was effectively reduced. Therefore, the use of user

Authorized licensed use limited to: NORTHWESTERN POLYTECHNICAL UNIVERSITY. Downloaded on May 20,2026 at 02:37:52 UTC from IEEE Xplore. Restrictions apply.

data in IoV edge caching schemes can effectively predict the popularity of content without compromising user privacy.

In high-mobility vehicle-roadside cooperation scenarios in IoV, current research is utilizing DRL methods to optimize edge caching strategies and reduce content transmission delay. For example, in [23], vehicle mobility is modeled as a Manhattan network, taking into account the probability of a vehicle choosing each direction and staying at an intersection. A multi-access edge intelligence and data transmission system is then designed using the Deep Deterministic Policy Gradient (DDPG) algorithm to minimize content delivery latency. In [24], the Markov Decision Process (MDP) is used to optimize the cache replacement process of edge nodes and is expanded to multi-agent networks. This aims to decrease content transmission costs and reduce traffic burden by promoting cooperative caching among multiple RSUs and MBSs. Similarly, in order address the issue of excessive latency caused by a significant increase in in-vehicle data traffic, [25] proposed an IoV e-commerce model that incorporates content distribution. This model utilizes a multi-agent DRL algorithm to solve the local caching decision problem for mobile vehicles, demonstrating exceptional performance in complex and heterogeneous IoV environments. Highly precise map data is crucial for smart connected vehicles. In response to the highly dynamic of vehicles and finite cache resources, a cooperative caching strategy utilizing FL and DRL was suggested in [26] to improve the accuracy of map information. In [27], blockchain and FL frameworks were applied to IoV environments, using a hybrid architecture of the blockchain comprising an authorized blockchain and a locally directed non-cyclic graph to increase the dependability of the model. The DRL algorithm was also used to implement node selection, improving the efficiency of the overall network system. While there has been significant research on edge caching for the IoV, little consideration has been given to the ability of the vehicle to collaborate with the various nodes at the edge. Most of the existing edge caching schemes consider cooperative caching among vehicles [28] or consider cooperative caching among RSUs [29], which do not fully utilize the caching resources among nodes. This study proposes a mobile-aware edge cooperative caching scheme (MAECCD) for the IoV that uses FL and DRL to address these issues. This scheme utilizes the user data to predict the content popularity of vehicles and regions while ensuring the security of user's data. Additionally, the DRL algorithm is used to update the cached content at each node.

III. System Model.

A. System Scenario

As shown in Fig. 1, we consider an urban roadway scenario consisting of an MBS, multiple RSUs, and multiple mobile vehicles under the range of the RSUs. A set of RSUs $S = \{s_1, s_2, s_3, \ldots, s_i\}$ are placed equidistant to the road side under the MBS coverage. Each RSU serves a set of vehicles, denoted as $V_i = \{v_1^i, v_2^i, v_3^i, \ldots, v_n^i\}$, where n is the total number of vehicles covered by the i-th RSU. It is assumed that the MBS possesses sufficient storage capacity to accommodate all the contents. However, RSUs and vehicles have limited storage capacities and

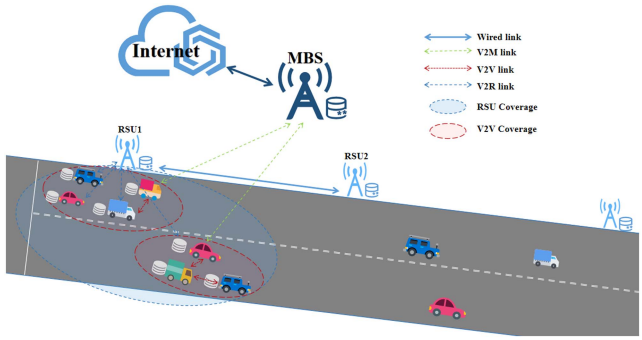


Fig. 1. System model.

can only cache a portion of the content. The cache capacity of RSUs can be represented by the set $C_S = \{c_{s_1}, c_{s_2}, c_{s_3}, \ldots, c_{s_i}\}$, while the cache capacity of vehicles can be represented by the set $C_{V_i} = \{c_{v_1^i}, c_{v_2^i}, c_{v_3^i}, \ldots, c_{v_n^i}\}$. And each vehicle stores a significant proportion of past data about the user, such as their past content requests and historical requests for content and the user's personal information. This personal information includes the user's ID, age, gender, and occupation.

We pre-cache a portion of the content into the RSU and the vehicle locally, based on the storage capacity size. There are five ways in which a vehicle can obtain content as it travels through the RSU area : 1) The user of the vehicle should first check the local cache and retrieve the requested content directly from there, if it is available. 2) If the requested content is not stored in the local cache, the vehicle can acquire it from other vehicles within the V2V communication range by establishing a V2V link. 3) If the requested content is not cached either in the local cache or in a vehicle within V2V communication range, it can be obtained from the local RSU via the V2R link. 4) If the content that is being requested is not stored in the local RSU cache, the vehicle can send a content request to the collaborating RSU via the current RSU, and acquire the content from the collaborating RSU via the local RSU relay. 5) If none of the above methods can retrieve the request content, the vehicle will retrieve it from the MBS.

B. Vehicle Mobility Model

The moving vehicle is modeled based on Poisson process. We take the first RSU at a road intersection as the Intersection RSU, and suppose that the quantity of vehicles $X_n(t)$ approaching the intersection RSU in the same direction over a period of time t is subject to a Poisson distribution with arrival rate λ_t:

$$P(X_n(t) = k) = \frac{(\lambda_t)^k}{k!} e^{-\lambda_t} \tag{1}$$

where k is the number of vehicles generated during a given time period, t.

The different vehicle speeds are independently and identically distributed and are obtained by generating a truncated Gaussian distribution. For example, at the intersection RSU the set of vehicles entering the scope area of the intersection RSU is defined as $V = \{v_1, v_2, v_3, \ldots, v_n\}$, and the set of cache capacities is $C = \{c_{v_1}, c_{v_2}, c_{v_3}, \ldots, c_{v_n}\}$, the set of its speeds is

Authorized licensed use limited to: NORTHWESTERN POLYTECHNICAL UNIVERSITY. Downloaded on May 20,2026 at 02:37:52 UTC from IEEE Xplore. Restrictions apply.

$u = \{u_1, u_2, u_3, \ldots, u_n\}$. The truncated Gaussian distribution limits the vehicle speed to a certain range, which better reflects the real dynamic environment of connected vehicles. Assuming that the speed of vehicles under the area of the intersection RSU remains constant during a specific duration, the probability density function can be expressed as follows.

$$f(u_n) = \begin{cases} \dfrac{e^{-\frac{1}{2\sigma_u^2}(u_n-\mu)^2}}{\sqrt{2\pi\sigma_u^2}\left(\mathrm{erf}\left(\frac{u_{\max}-\mu}{\sigma_u\sqrt{2}}\right)-\mathrm{erf}\left(\frac{u_{\min}-\mu}{\sigma_u\sqrt{2}}\right)\right)}, \\ \qquad u_{\min} \le u \le u_{\max}, \\ 0, \qquad \text{otherwise}. \end{cases} \tag{2}$$

where ${\sigma_u}^2$ is the variance, μ is the mean, $u_{\min}$ and $u_{\max}$ are the minimum and maximum upper limits of the vehicle speed, respectively, and $\mathrm{erf}()$ is the Gaussian error function.

C. Communication Model

Inter-vehicle communication is realized through specialized short distance communication technology, where vehicles are able to communicate with other vehicles that meet the communication range requirements when requesting a document. Different RSUs communicate with each other using a wired direct link, and since the vehicle is in a moving state, vehicle location changes all the time, so the rate of communication between the vehicle and the RSUs and the MBS varies in different time periods. According to [30], we model the channel gain of the vehicle v_n as:

$$h_n\left(d\left(x, v_n\right)\right) = \partial_n\left(d\left(x, v_n\right)\xi_n\left(d\left(x, v_n\right)\right), x = \{V, S, M\}\right. \tag{3}$$

where n is the number of vehicles under the coverage of the intersection RSU, $x = \{V, S, M\}$ represents vehicles, RSUs and MBSs, respectively, $\mathrm{d}(x, v_n)$ is the distance between vehicle v_n and other vehicles, RSUs and MBSs, and $\partial_n(d(x, v_n))$ and $\xi_n(d(x, v_n))$ are the path loss and shadow fading between vehicles and other communication devices, respectively. Based on Shannon's theorem, the transmission rates of vehicle v_n and vehicle v_k are calculated [31]:

$$R_{v_n, v_k} = B\log_2\left(1 + \frac{p_V h_n\left(d\left(v_k, v_n\right)\right)}{\sigma^2}\right) \tag{4}$$

where B is the transmission bandwidth, p_V is the transmission power for V2V communication between vehicles, and σ^2 is the noise power.

Similarly, the transmission rate between vehicle v_n and the local RSU can be calculated as:

$$R_{v_n, S} = B\log_2\left(1 + \frac{p_S h_n\left(d\left(S, v_n\right)\right)}{\sigma^2}\right) \tag{5}$$

where p_S is the transmit power of the local RSU.

Similarly, the transmission rate between vehicle v_n and remote MBS is calculated as:

$$R_{v_n, M} = B\log_2\left(1 + \frac{p_M h_n(d(M, v_n)}{\sigma^2}\right) \tag{6}$$

where p_M is the transmit power of the MBS.

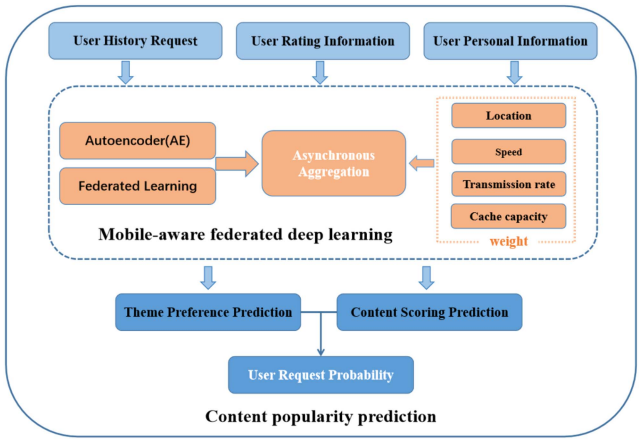


Fig. 2. Content popularity prediction framework.

IV. A Popularity Prediction Method Based on Asynchronous FL

In this section, we present a popularity prediction method (shown in Fig. 2) that is specifically designed for the dynamic IoV environment. Our method takes into account the user's historical request information, content theme preferences and personal information, while also ensuring the protection of their privacy through the incorporation of FL. The model is updated in the way of asynchronous aggregation, and the influence of vehicle speed, location, transmission rate and vehicle cache capacity is considered to improve the accuracy of the model. Compared to traditional content popularity prediction methods, our proposed approach is able to more accurately capture the current content heat of the vehicles in the area, making it better suited for the highly mobile and dynamic nature of the connected IoV environment.

A. Mobile-Aware Federated Deep Learning

We use a combination of AutoEncoder and FL to extract higher-order features from user information, which we then use for predicting popularity. Considering the mobility of vehicles, we employ asynchronous aggregation to update the global model. The flow of this part is shown in Fig. 3(a) and can be divided into the following five steps.

1) Vehicle Selection: Because of the restricted extent of RSU coverage, if the vehicle speed is too fast it may affect the successful completion of the model training and popularity prediction. Therefore, the vehicle selection should be performed before the training. During vehicle selection, the vehicle first sends its information regarding speed and location to the current RSU. The RSU then filters the vehicles by calculating the vehicle staying time, which is defined as the time from when a vehicle sends its speed and location information to when it drives out of the local RSU coverage. The staying time for n-th vehicle is calculated as [21].

$$T_{v_n}^{staying} = (D - P_{v_n})/u_n \tag{7}$$

where D is the diameter of the area covered by the local RSU, P_{v_n} is the distance from the entrance position of the intersection

Authorized licensed use limited to: NORTHWESTERN POLYTECHNICAL UNIVERSITY. Downloaded on May 20,2026 at 02:37:52 UTC from IEEE Xplore. Restrictions apply.

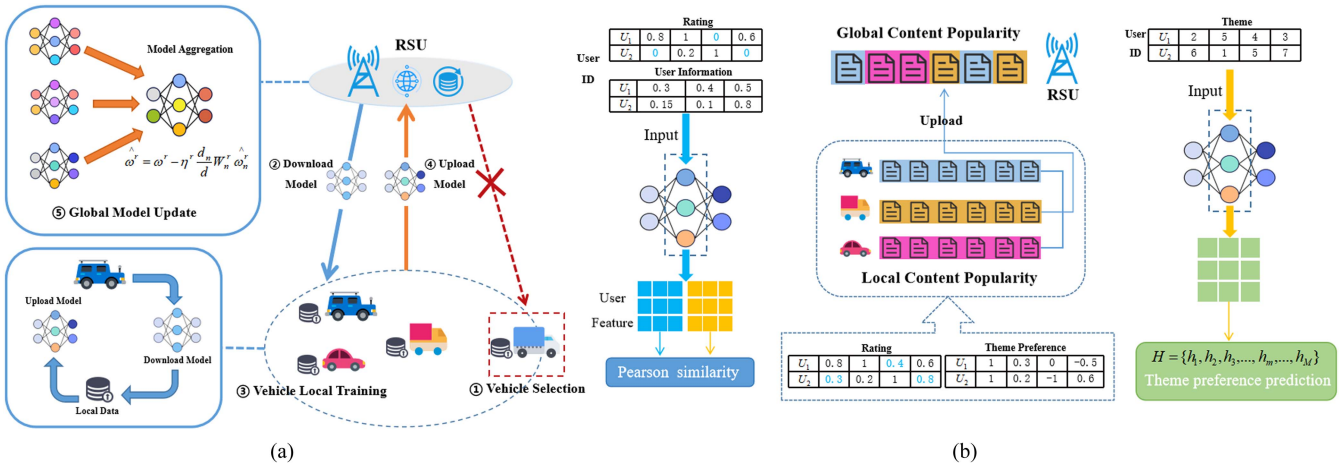


Fig. 3. (a) Federated deep learning process of mobile-aware. (b) Popularity prediction process.

RSU to the vehicle's current position, and u_n is the vehicle's speed. A vehicle will be chosen for training if its staying time satisfies the condition $T_V^{staying} > T_{train} + T_{\inf}$, where T_{train} is the time taken for average training and T_{inf} is the time taken for inference.

2) Model Download: In the first training, the Intersection RSU will build a global model ω based on the AutoEncoder and distribute the global model to the vehicles selected for training. For the vehicles selected in the next round, instead of generating a global model, the intersection RSU will distribute the trained global model from the last round directly to the currently selected vehicles. Continuing to train on the global model from the previous round not only improves the speed at which the model is trained, but also results in a more accurate model.

3) Model Training: During this stage, the vehicles selected for training in the r-th round have already downloaded the global model ω^r from the intersection RSU. We use the set $\omega_V^r = \{\omega_1^r, \omega_2^r, \omega_3^r, \ldots, \omega_V^r\}$ to represent the models downloaded by all vehicles. Each vehicle utilizes its own local user data to get involved in learning. However, the size of the local user data may vary among vehicles, and this can affect the training of the model. To account for this, we use the set $d = \{d_1, d_2, d_3, \ldots, d_n\}$ to represent the size of the training dataset for each vehicle.

During the local training of each vehicle, a portion of the local data, with a size of e_n, is selected data in each iteration to be used in the training process. The loss function, $l(\omega_V^r; x)$, can be calculated as:

$$l(\omega_n^{\mathrm{r}}; x) = \frac{1}{\mathrm{e_n}} \sum_{x \in e_n} (x - \hat{x})^2 \tag{8}$$

x is the input of the encoder and $\hat{x}$ is the output of the encoder. The first round of training $\omega_V^r = \omega^r$.

The vehicle local training model is updated as:

$$\hat{\omega_n^r} = \omega_n^r - \eta_n^r \nabla l(\omega_n^r; x) \tag{9}$$

η_n^r is the learning rate in the local training loop iterations of the vehicle.

For vehicle local training our objective is to minimize its loss function. For overall federated deep learning our goal is to minimize the global model loss function $l(\omega^r)$. That is:

$$\min_{\omega^r} l(\omega^r) = \sum_{n=1}^{V} \frac{d_n}{d} l(\omega_n^r; x) \tag{10}$$

where d_n is the training data size of the $n\text{-}th$ vehicle and d is the overall data size for all vehicles.

4) Model Upload: After finishing the local training vehicle uploads its model parameters ω_V^r to the RSU, which then begins updating the global model with the model parameters from the vehicle.

5) Global Model Update: After the vehicle local models are successfully uploaded, intersection RSU will calculate the weighted sum of all vehicle local model parameters ω_V^r to update the global model. The traditional FL model uses synchronous aggregation, which requires all vehicle local models to be uploaded successfully before aggregation and finally updating the global model. However, in actual training, some vehicles may have driven out of the intersection RSU without completing the local training for some reasons, which makes the local training model unable to be uploaded to the intersection RSU, which is not conducive to the updating of the global model as well as the subsequent prediction of the popularity. Therefore, we use asynchronous aggregation when updating the global model, and the global model will be updated once after a vehicle successfully uploads the local model. In this way, even if a vehicle fails to upload the local model due to driving out of the intersection RSU, it will not affect the update of the global model, which is more suitable for high-speed vehicle environment. When the $n\text{-}th$ vehicle completes the model upload, the global model is updated as:

$$\hat{\omega^r} = \omega^r - \eta^r \frac{d_n}{d} \hat{\omega_n^r} \tag{11}$$

where η^r is the learning rate.

To further enhance the accuracy of the model, weighting factors such as vehicle location, vehicle speed, vehicle cache capacity and transmission rate should be considered during asynchronous aggregation, in addition to the size of the vehicle's local training data. Specifically, if the vehicle is far from the exit

Authorized licensed use limited to: NORTHWESTERN POLYTECHNICAL UNIVERSITY. Downloaded on May 20,2026 at 02:37:52 UTC from IEEE Xplore. Restrictions apply.

and relatively slow, the vehicle will have a longer dwell time to participate in training within the intersection RSU coverage and should therefore be given a larger weight. In addition, after the model training and popularity prediction is completed, the data needs to be stored locally in the vehicle and in the RSUs, so the size of the cache capacity for different vehicles and the transmission rate between the vehicle and the local RSUs are also important. So the larger the vehicle cache capacity and the higher the transmission rate between the vehicle and the RSU, the more weight should be given to this. Further the global model update is rewritten as:

$$\hat{\omega^r} = \omega^r - \eta^r \frac{d_n}{d} W_n^r \hat{\omega_n^r} \tag{12}$$

W_n^r is the vehicle weight, which is calculated as:

$$W_{\mathrm{n}}^r = \alpha_1 \frac{T_{v_n}^{staying}}{\max_{n\in V} T_{v_n}^{staying}} + \alpha_2 \frac{R_{v_n,S}}{\max_{n\in V} R_{v_n,S}} + \alpha_3 \frac{c_{v_n}}{\max_{n\in V} c_{v_n}} \tag{13}$$

$\alpha_1, \alpha_2, \alpha_3$ is the staying time, transmission rate and cache capacity factor, respectively, which can be set according to the actual situation to satisfy $\alpha_1 + \alpha_2 + \alpha_3 = 1$. $T_{v_n}^{staying}$, $R_{v_n,S}$ and c_{v_n} are the staying time, transmission rate, and cache capacity of the $n\text{-}th$ car, respectively.

After the global model aggregation is completed, the intersection RSU distributes the updated global model to the vehicles. The vehicles perform popularity prediction after receiving the global model.

B. Content Popularity Prediction

Traditional popularity is mostly based on all user requests in a region to obtain the overall content popularity ranking, and does not change over a period of time, the content popularity obtained in this way is not applicable to the high mobility of the IoV environment. Moreover, the traditional way is to get the overall popularity of a region, which does not accurately reflect the content that users are really interested in, so it is important to accurately grasp the content preferences of different users in order to achieve a more effective popularity of the content and thus improve the region's caching hit rate. The overall process of popularity prediction is shown in Fig. 3, which is divided into two parts. Fig. 3(a) shows the federated deep learning process of mobile-aware, in which vehicles download the global model from the intersection RSU, train the model using local data, and upload the model training parameters to update the global model, as described in the previous section. Fig. 3(b) shows the popularity prediction process, in which vehicles download the pre-trained AutoEncoder model (i.e., the global model) from the RSU, use the AutoEncoder to predict content ratings and preference for themes. Specifically, we use the user history rating, user information and the theme of the file in the history through the AE model, and use the obtained higher-order characteristics to predict the user's rating of the unrated content and the preference for different themes respectively. Finally, we obtain the content popularity for each vehicle, and we obtain the global popularity for the current RSU coverage area through comprehensive calculation. In this section, we will describe the (b) part of the process in detail.

1) Content Scoring Prediction: Different vehicles contain data about different users, including user history request information and user personal information. Among other things, the user history request information contains what content has been requested by the user, the ratings of that content, and the subject matter of that content. Let us take vehicle v_n as an example, with the number of users $U = \{1, 2, 3, \ldots, u\}$ in the vehicle and the full set of contents $F = \{1, 2, 3, \ldots, f\}$. The vehicle dataset is first preprocessed to obtain a matrix of user content ratings $\mathfrak{R}_{U\times F}$ and a matrix of user information $\mathfrak{I}_{U\times A}$. Define the value $r_{u,f} \in [0, 1]$ of the user content rating matrix as the user's rating of the content, $r_{u,f}$ larger value indicates that the user likes the content more, and $r_{u,f} = 0$ indicates that the user did not rate the content. The values $a_{u,k}(k \in \{1, 2, 3\})$ of the user information matrix represent the age, gender and occupation of the user, respectively. In real life, it is impossible for users to rate all content, and for unrated content we cannot determine the user's preference for it. Therefore, it is necessary to predict users' unrated content to better reflect their content preferences. Here we utilize a user-based collaborative filtering (CF) algorithm to derive social relationships among users as a way to predict users' preferences for unrated content [32].

The CF algorithm utilizes other users who are highly similar to the target object to predict the ratings, so first we need to obtain similar users to the target object. Here we consider two aspects to measure the similarity between users, i.e., user ratings of content and user information. We define the user similarity $\theta_{i,j}$ (j is a user other than user i) as:

$$\theta_{i,j} = \frac{\sum_{f=1}^{F}(r_{i,f} - \overline{r_i})(r_{j,f} - \overline{r_j})}{\sqrt{\sum_{f=1}^{F}(r_{i,f} - \overline{r_i})^2}\sqrt{\sum_{f=1}^{F}(r_{j,f} - \overline{r_j})^2}} \times \frac{\sum_{k=1}^{A} a_{i,k} \cdot a_{j,k}}{\sqrt{\sum_{k=1}^{A} a_{i,k}^2}\sqrt{\sum_{a=1}^{A} a_{j,k}^2}} \tag{14}$$

where $r_{i,f}$ and $r_{j,f}$ represent the ratings of user i and user j on content f, respectively,$\overline{r_i}$ and $\overline{r_j}$ represent the average ratings of user i and user j on all content, respectively,and $a_{i,k}$ and $a_{j,k}$ represent the personal information of user i and user j, respectively.

We set a similarity threshold θ_{th} and filter the users whose similarity meets the requirement to obtain the set U_{sim} of similar users.

$$U_{sim} = \{j | \theta_{i,j} > \theta_{th}, j \neq i\} \tag{15}$$

Finally, we use the rating information of similar users to infer the target user's rating of unrated content:

$$r_{i,f} = \begin{cases} \overline{r}_i + \frac{\sum_{j\in U_{sim}} \theta_{i,j}(r_{j,f} - \overline{r_{u_j}})}{\sum_{j\in U_{sim}} \theta_{i,j}} & \text{if} \quad r_{i,f} = 0 \\ r_{i,f} & \text{if} \quad r_{i,f} \neq 0 \end{cases} \tag{16}$$

2) Prediction of Theme Preferences: The user's rating of the content can reflect the user's preference for the content, but the rating does not fully reflect the user's preference,

Authorized licensed use limited to: NORTHWESTERN POLYTECHNICAL UNIVERSITY. Downloaded on May 20,2026 at 02:37:52 UTC from IEEE Xplore. Restrictions apply.

we can more accurately grasp the user's preference by predicting the user's preference for different themes [33]. Here we divide all the contents into M themes, i.e., the set $H = \{h_1, h_2, h_3, \ldots, h_m, \ldots, h_M\}$. The relation function between theme h_m and content f can be expressed as follows:

$$\rho_{h_m,f} = \begin{cases} 1 & \text{if} \quad f \in h_m \\ 0 & \text{if} \quad f \notin h_m \end{cases} \tag{17}$$

Our ultimate goal is to get the degree of user's preference for different themes and to predict the user's preference more accurately. Here we utilize the mutual correlation function in information theory to measure the user's preference for different themes [34], the theme preference function ϕ_{u,h_m}:

$$\phi_{u,h_m} = I(X(h_m); Y(u)) = \log \frac{P(X(h_m)|Y(u))}{P(X(h_m))} \tag{18}$$

where is $I(X(h_m); Y(u))$ mutual information, $X(h_m)$ is the set of all contents containing theme h_m, $Y(u)$ is the set of contents of user's historical requests, $P(X(h_m)|Y(u))$ is the probability that the set of contents of user's requests contains theme h_m, and $P(X(h_m))$ is the probability that the entire database contains theme h_m.

Therefore, the relationship function $\rho_{h_m,f}$ and the theme preference function ϕ_{u,h_m} can be used to define the user's interest function for the content:

$$\phi_{u,f} = \frac{\sum_{m=1}^{M} \rho_{h_m,f} \phi_{u,h_m}}{\sqrt{\sum_{m=1}^{M} \rho_{h_m,f}^2} \sqrt{\sum_{m=1}^{M} \phi_{u,h_m}^2}} \tag{19}$$

where the result of the function w takes values in the range [−1,1], the closer the result is to 1 the more interested the user is in the content, and likewise the more likely the user is to request the content.

3) Content Request Probability Prediction: Through the CF algorithm we obtained the users' ratings for unrated content and the users' preferences for different content using the users' historical request data and content theme categorization. In the following, we will consider a combination of the user ratings and content preferences to predict the probability of request content.

In order to enhance the disparity in the probability of user requests for different contents more significant, we first construct a scoring discrepancy function, defined as:

$$g_{u,f}^{r} = \frac{r_{u,f} - r_u^{\min}}{r_u^{\max} - r_u^{\min}} \tag{20}$$

where $r_u^{\min}$ and $r_u^{\max}$ are the minimum and maximum values in the user content ratings, respectively.

Thus, the probability of a content request predicted by a user's content score can be calculated as:

$$p_{u,f}^{r} = \frac{r_{u,f}^{\mathcal{E} g_{u,f}^{r}}}{\sum_{f=1}^{F} r_{u,f}^{\mathcal{E} g_{u,f}^{r}}} \tag{21}$$

$\mathcal{E}$ is used to adjust for differences between user preferences.

Algorithm 1: Content Popularity Prediction Algorithm.

Initialization: Global model ω;
1: **for** round $r = 1 : r_{\max}$ **do**
2: **for** each selected v_n in parallel **do**
3: Download global model ω from the RSU;
4: Calculate loss function $l(\omega_V^{\mathrm{r}}; x)$ according to (8);
5: $\hat{\omega_n^r} \leftarrow \omega_n^r$ based on (9);
6: Upload the $\hat{\omega_n^r}$ to the RSU;
7: **end for**
8: Calculate the weight of the asynchronous aggregation W_{n}^r according to (13);
9: $\hat{\omega^r} \leftarrow \omega^r$ based on (12);
10: **return** $\hat{\omega^r}$
11: **end for**
Input: User content scoring matrix $\mathfrak{R}_{U\times F}$;User information matrix $\mathfrak{I}_{U\times A}$;Divide the database into M themes,where $H = \{h_1, h_2, h_3, \ldots, h_m, \ldots, h_M\}$;
12: **for** each selected v_n in parallel **do**
13: **for** $i = 1 : U$ **do**
14: **for** $j = 1 : U$ **do**
15: Compute the value of $\theta_{i,j}$ based on (14);
16: $U_{sim} = \{j|\theta_{i,j} > \theta_{th}, j \neq i\}$;
17: **end for**
18: Compute unrated content based on (16);
19: **for** $m = 1 : M$ **do**
20: **for** $f = 1 : F$ **do**
21: Compute the value of ϕ_{i,h_m} based on (18);
22: Compute the value of $\phi_{i,f}$ based on (19);
23: **end for**
24: **end for**
25: **end for**
26: Calculate $O_f^{v_n}$ according to (25);
27: Calculate O_f according to (26);
28: **end for**

Similarly, we can get the differences in user preferences for different content:

$$g_{u,f}^{\phi} = \frac{\phi_{u,f} - \phi_u^{\min}}{\phi_u^{\max} - \phi_u^{\min}} \tag{22}$$

where $\phi_u^{\min}$ and $\phi_u^{\max}$ are the lowest and highest scores in user content preferences, respectively.

The content request probability predicted by theme preferences can be calculated as:

$$p_{u,f}^{\phi} = \frac{r_{u,f}^{\mathcal{E} \mathrm{g}_{u,f}^{\phi}}}{\sum_{f=1}^{F} r_{u,f}^{\mathcal{E} \mathrm{g}_{u,f}^{\phi}}} \tag{23}$$

Therefore, the probability of a content request that combines the user's content score and the user's theme preference is calculated as:

$$p_{u,f} = \varphi p_{u,f}^{r} + (1 - \varphi) p_{u,f}^{\phi} \tag{24}$$

where φ is the weight of different content request probabilities.

Finally, we compute the local popularity of vehicle v_n based on the probability of content being requested from each user in

Authorized licensed use limited to: NORTHWESTERN POLYTECHNICAL UNIVERSITY. Downloaded on May 20,2026 at 02:37:52 UTC from IEEE Xplore. Restrictions apply.

the vehicle:

$$O_f^{v_n} = \frac{\sum_{u=1}^{U} p_{u,f}}{\sum_{u=1}^{U}\sum_{f=1}^{F} p_{u,f}} \tag{25}$$

After obtaining the local prevalence of all vehicles, they upload this information to the intersection RSU. The RSU then calculates the global popularity in the region.

$$O_f = \frac{\sum_{n=1}^{V} O_f^{v_n}}{\sum_{n=1}^{V}\sum_{f=1}^{F} O_f^{v_n}} \tag{26}$$

Eventually, we were able to obtain the local content popularity of each vehicle and the global content popularity under RSU coverage.

V. Mobile-Aware Cooperative Cache Scheme Based on DRL(MAECCD)

With the above work, we conduct the prediction of the popularity within the intersection RSU coverage and get the content popularity of the vehicles under the intersection RSU coverage for that time period. In order to ensure the validity of the popularity and match with the vehicles, the predicted popularity will follow the vehicles cached to the next RSU.

To fully utilise the finite caching resources of vehicles and RSUs in edge caching environment, we will utilize the Dueling DQN algorithm in DRL to update the contents of the RSUs and vehicle caches while the vehicle is in motion. This will allow us to make the most of the limited storage resources available. In addition to the vehicle and local RSUs, nearby RSUs also have caching capabilities in the edge environment. To fully utilize the cache resources among nodes, we have designed a DRL-based cooperative caching strategy to enhance the cache utilization in edge devices and improve the responsiveness of Telematics services. The details of this cooperative caching scheme are outlined below.

A. Content Caching

After the popularity prediction is completed, the RSU will cache the top-ranked global popularity content based on its cache capacity. Similarly, the vehicle will cache the top-ranked local popularity content. When a vehicle, which has already predicted its popularity at an intersection RSU, drives from the current RSU to another, the cache of the next RSU will be updated according to the popularity of the current vehicle. However, the local cache of the vehicle will remain unchanged. This approach ensures that the RSU's cache is regularly updated at different time intervals, maintaining the timeliness of popular content within its coverage area and accommodating the mobility of the vehicle.

B. Cache Update

In this section, we will utilize the Dueling DQN algorithm to update the RSU and vehicle caches, as shown in Fig. 4.

1) Vehicle Clustering: To decrease the signaling overhead of V2V broadcast communication, and to simplify the user request and content distribution processes in the system model, we will first cluster the vehicles. We will consider the weights of vehicle speed, position, and V2V communication distance, and use a fast clustering method applicable to the IoV.

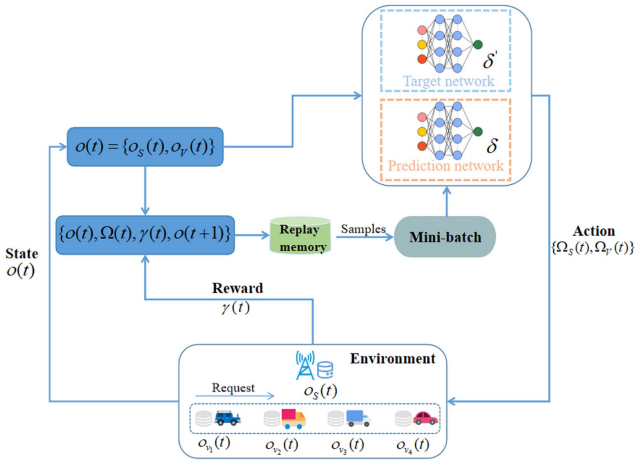


Fig. 4. Flowchart of Dueling DQN.

The RSU receives the vehicle information, denoted as $I = \{I_1, I_2, I_3, \ldots, I_n\}$, and begins the process of vehicle clustering. I_n contains speed and position information of the n-th vehicle. The first vehicle is designated as the first node in a new cluster, and its assignment to an existing cluster is determined by calculating its similarity to the other clusters. This is done using the Euclidean distance metric, as described Z [35], which is computed as follows.

$$SIM(v_n, Z) = \frac{\sum_{j=1}^{z} dist(v_n, v_j)}{z} \tag{27}$$

where z is the quantity of vehicles in cluster Z, and $dist(v_n, v_j)$ refers to the Euclidean distance between vehicle v_n and vehicle v_j within the cluster.

We use the range of V2V communication as a threshold. If multiple clusters meet the threshold requirement, the vehicles are assigned to the cluster with the highest similarity. Otherwise, the vehicles form a new cluster. This method allows for fast clustering of vehicles without prior knowledge of the number of clusters, and it can accommodate clusters of various shapes, making it suitable for the IoV environment.

2) The DRL Framework Consists of Three Main Components: State, Action and Reward:

a) State: We take the contents of the RSU cache and the local cache contents of all vehicles in its coverage region as the state $o(t) = \{o_S(t), o_V(t)\}$,where $o_S(t)$ and $o_V(t) = \{o_{v_1}(t), o_{v_2}(t), o_{v_3}(t), \ldots, o_{v_n}(t)$ are the caches of the local RSU and all vehicles at moment t. For the cached content in the state are obtained in descending order of predicted content popularity.

b) Action: The designed action $\Omega(t) = \{\Omega_S(t), \Omega_V(t)\}$ consists of two parts, $\Omega_S(t) \in \{0, 1\}$ is used to indicate whether or not to do an update to the cache content in the RSU, and $\Omega_V(t) \in \{0, 1\}$ is used to indicate whether or not to do an update to the vehicle's local cache. Specifically, if $\Omega_S(t) = 1$, N_S contents will be selected at random from the global popularity uncached content to replace the less popular content

Authorized licensed use limited to: NORTHWESTERN POLYTECHNICAL UNIVERSITY. Downloaded on May 20,2026 at 02:37:52 UTC from IEEE Xplore. Restrictions apply.

in the RSU cache. The cached content will then be reordered according to the global popularity to obtain the state $o_S(t+1)$. On the other hand, if $\Omega_S(t)=0$, no changes will be made to the cached content of the RSU. The vehicle will decide whether to replace the cached content based on the request information. If $\Omega_{v_n}(t)=1$, N_{v_n} contents will be randomly selected from the unhit contents to replace the less popular contents stored in the local cache. The local cache will then be reordered according to the local popularity to get the state $o_{v_n}(t+1)$; if $\Omega_{v_n}(t)=0$, the contents of the vehicle's local cache remain unchanged.

c) Reward: Our objective is to minimize the transmission delay of the requested content and improve the user's experience. Therefore, our reward function $\gamma(t)$ is based on the content transmission delay. The delay of obtaining content through various nodes is calculated as follows, which will compute the delay τ_V^f of obtaining the content f from various nodes (presuming that the content is of the same size as F_f).

If the requested content f is cached locally, the content transmission delay t is 0, i.e., $\tau_{v_n}^f=0$.

If the content f that has been requested is cached in a vehicle v_j within a cluster that satisfies the V2V communication requirements, the content transmission delay is computed as:

$$\tau_{v_n,v_j}^f=\frac{F_f}{R_{v_n,v_j}} \tag{28}$$

where R_{v_n,v_j} is the transmission rate of the two vehicles, which is obtained from (4).

If the content f that has been requested is stored in the current RSU, the delay is calculated as:

$$\tau_{v_n,S}^f=\frac{F_f}{R_{v_n,S}} \tag{29}$$

where $R_{v_n,S}$ is the transmission rate between the vehicle and the RSU, obtained from (5).

We take the next RSU along the vehicle's traveling direction as the collaborating RSU, and if the content f that has been requested is cached in the collaborating RSU, the collaborating RSU will send the content to the vehicle via the local RSU, whose content transmission delay is calculated as:

$$\tau_{v_n,S'}^f=\frac{F_f}{R_{v_n,S}}+\frac{F_f}{R_{R,R'}} \tag{30}$$

where $R_{R,R'}$ represents the transmission rate between the current RSU and the collaborating RSUs, with a fixed value for the rate over a wired connection.

If none of the aforementioned nodes are able to obtain the requested content, they will need to retrieve it from the MBS. Therefore, the calculation for the content transmission delay is as follows.

$$\tau_{v_n,M}^f=\frac{F_f}{R_{v_n,M}} \tag{31}$$

where $R_{v_n,M}$ refers to the transmission rate between the vehicle and the MBS, which is calculated using (6). The reward function, indicated as $\gamma_{v_n}^f(t)$, is constructed according to the transmission delay of the vehicle v_n in obtaining content f from various nodes.

$$\gamma_{v_n}^f(t)=\begin{cases}\chi_0 & from \quad v_n\\ \chi_1 e^{-\lambda_1\tau_{v_n,v_j}^f} & from \quad v_j\\ \chi_2 e^{-\lambda_2\tau_{v_n,S}^f} & from \quad RSU\\ \chi_3 e^{-(\lambda_2\tau_{v_n,S}^f+\lambda_3\tau_{v_n,S'}^f)} & from \quad RSU'\\ \chi_4 e^{-\lambda_4\tau_{v_n,M}^f} & from \quad MBS\end{cases} \tag{32}$$

where $\lambda_1+\lambda_2+\lambda_3+\lambda_4=1$ and $\lambda_1<\lambda_2<\lambda_3<<\lambda_4$. χ represents the coefficient of variation. This is a measure used to account for variations in rewards.

d) DRL algorithm process: Dueling DQN includes two network structures, the prediction network and the goal network [36]. The prediction network is used to evaluate the current state and action and to calculate the Q-value; the target network is utilized to obtain the ideal Q-value function. Every network consists of two components: the advantage function and the state value function.

First, we randomly initialize the parameters for the prediction network, denoted as δ, and the target network, denoted as δ'. Once the state $o(t)$ is entered into the network, the prediction network will calculate the Q-value for different actions.

$$Q(o(t),\Omega;\delta)=Y_o(o(t);\delta)+\{Y_\Omega(o(t),\Omega;\delta)-\mathrm{E}[Y_\Omega(o(t),\Omega;\delta)]\} \tag{33}$$

Algorithm 2: Mobile-Aware Collaborative Caching Based on DRL(MAECCD).

Input: Vehicle information I_V,Contents requested by all vehicles at slot $t\in\{1,2,3,\ldots,T\}$,Popular contents for vehicle $O_f^{v_n}$,Popular contents in the RSU coverage area O_f;

Initialization: Parameters of the prediction network δ, Parameters of the target network δ',Replay buffer;

```
for epsiode = 1 : MAXEPISODE do
  Vehicles cache popular content from O_f^{v_n};
  RSU caches popular content from O_f;
  for step = 1 : MAXSTEP do
    Observe the state o(t);
    Calculate the Q-value of prediction network according to (33);
    Selection of actions {Ω_S(t), Ω_V(t)} by ε-greedy strategy;
    Acquire state o(t+1) following the execution of action Ω_(t);
    Store tuple {o(t), Ω(t), γ(t), o(t+1)} in Replay buffer;
    Adjust parameters of the prediction network based on (34)–(38): δ ← δ − η_δ ∇_δ L(δ);
    if the slot is T then
      Update parameters of the target network: δ' ← δ;
    end if
  end for
end for
```

Authorized licensed use limited to: NORTHWESTERN POLYTECHNICAL UNIVERSITY. Downloaded on May 20,2026 at 02:37:52 UTC from IEEE Xplore. Restrictions apply.

where $Y_o(o(t);\delta)$ is the state value function and $Y_\Omega(o(t),\Omega;\delta)$ is the advantage function.

After calculating the Q-value, the action $\Omega(t)$ is selected and executed using the $\varepsilon - greedy$ strategy. Next, the reward value $\gamma(t)$ for that action is calculated and a new state $o_S(t+1)$ is obtained. The tuple $\{o(t),\Omega(t),\gamma(t),o(t+1)\}$ is then stored into the replay buffer.

The same target network computes Q' under different actions:

$$Q'(o(t),\Omega;\delta') = Y'_o(o(t);\delta') + \{Y'_\Omega(o(t),\Omega;\delta') - \mathrm{E}[Y'_\Omega(o(t),\Omega;\delta')]\} \tag{34}$$

The target value function is then calculated:

$$\hat{\mathrm{Y}} = \gamma(t) + \zeta \max Q'(o(t),\Omega;\delta') \tag{35}$$

where ζ is the discount factor. The loss function is calculated as follows.

$$L(\delta) = (\hat{\mathrm{Y}} - Q(o(t),\Omega;\delta))^2 \tag{36}$$

The gradient of the loss function is computed as:

$$\nabla_\delta L(\delta) = (\hat{\mathrm{Y}} - Q(o(t),\Omega;\delta))\nabla_\delta Q(o(t),\Omega;\delta) \tag{37}$$

The parameters of the prediction network δ at the next moment are updated to:

$$\delta \leftarrow \delta - \eta_\delta \nabla_\delta L(\delta) \tag{38}$$

where η_δ is the learning rate of the prediction network.Similarly, the target network parameters δ' will be updated periodically:

$$\delta' \leftarrow \delta \tag{39}$$

Finally, we utilize DRL to update the cached content in the active caching strategy, and find the optimal cache update scheme by designing the content transmission delay cost function to minimize the latency in transmitting content. The mobile-aware collaborative caching algorithm dynamically updates the vehicle's local cache content and local RSU cache content while the vehicle is in motion, which further improves the performance of IoV and enhances the quality of user experience.

e) Complexity of algorithm: We consider the computational complexity in terms of neural network structure, state space dimension, action space dimension and custom reward function.For deep neural networks, the time complexity of forward propagation is mainly related to the network structure, and the time complexity of this part is $O(L \cdot l^2)$, where L is the number of network layers and l is the number of nodes per layer. Considering that the output layer is directly related to the action dimension, the complexity of forward propagation is modified to $O(L \cdot l^2 + l \cdot A)$, where A is the action space dimension. In the process of experience replay, Dueling DQN will randomly select samples of size b for training, assuming that the number of training rounds is T, then the total time complexity is $O(T \cdot b \cdot (L \cdot l^2 + l \cdot A))$. According to the definition of the reward function, the computational complexity of this part is $O(1)$, and it does not affect the overall time complexity. The space complexity is mainly related to the network parameters, the experience replay pool size, and the action dimension, and can be expressed as $O(L \cdot l^2 + B + l \cdot A)$, where B is the experience replay pool size. The state space dimension directly affects the size of the experience replay pool and thus affects the space complexity, but the state we define does not have a large impact on the complexity. In particular, we adopt a dual network structure, which increases the number of parameters, which will increase some space complexity, but can provide a more stable training process. And this part has a relatively limited impact on the time complexity, which is more suitable for time-sensitive applications.

TABLE I
SIMULATION PARAMETERS

Parameter	Value	Parameter	Value
System Parameters			
B	540KHz	p_v	3dBm
σ^2	-114dBm	p_s	30dBm
$u_{\min}/u_{\max}$	50km/h-60km/h	p_M	43dBm
D	1000m	$\mathcal{E}$	5
φ	0.3	T_{inf}	0.5s
AutoEncoder			
number of hidden layers	1	number of hidden units	100
η^r	0.01	α_1,α_2,α_3	0.4,0.3,0.3
Dueling DQN			
η_δ	0.01	λ_1,λ_2,λ_3,λ_4	0.001,0.01,0.39,0.5999
ζ	0.99		
advantage layer		state-value layer	
number of hidden layer	1	number of hidden layer	1
number of hidden units	128	number of hidden units	128

TABLE II
HARDWARE PARAMETERS

CPU	Quad-core processor (e.g., Intel Core i5-XXXX or AMD Ryzen 5)
GPU	NVIDIA GTX 1050 or equivalent performance GPU
RAM	8GB DDR4 RAM, typically running at 2400 MHz or higher
SSD	128GB SSD or higher

VI. SIMULATION RESULTS AND PERFORMANCE ANALYSIS

A. Simulation Settings

The communication between the vehicle, RSU and MBS in the simulation model of this paper utilizes the 3GPP cellular V2X (V-V2X) architecture. The parameters have been set in accordance with the 3GPP standard [30]. Unless otherwise stated, the vehicle density is 15 vehicles/km. The data used in the experiments to calculate the popularity and train the DRL model comes from the real dataset MovieLens 1M from the MovieLens website. It contains ratings of 3952 movies from 6040 anonymous users, each of whom has rated at least 20 movies. MovieLens 1M also includes data on users' IDs, ages, and occupations, as well as genre information for each movie [37]. We distribute the rated user information among participating vehicles as local data. Each vehicle then randomly selects 98% of the local data as the training set and the remaining 2% as the test set during training. The simulation parameters and hardware minimum parameters are shown in Tables I and II:

B. Performance Evaluation

Cache hit rate, content transmission delay, transmission energy consumption and throughput are utilized as performance evaluation indicators. The cache hit rate and content transmission delay are defined as the average value of the hit rate and the

Authorized licensed use limited to: NORTHWESTERN POLYTECHNICAL UNIVERSITY. Downloaded on May 20,2026 at 02:37:52 UTC from IEEE Xplore. Restrictions apply.

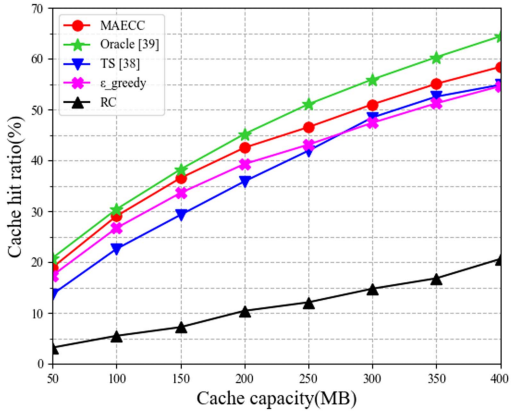


Fig. 5. Hit rate with different cache capacities.

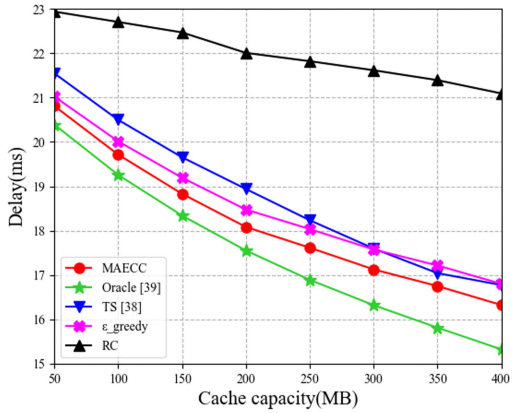


Fig. 6. Content transmission delay with different cache capacities.

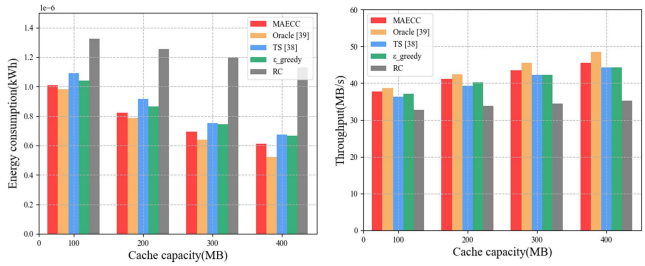


Fig. 7. Comparison of transmission energy consumption and throughput under different cache capacities. (a) Energy consumption. (b) Throughput.

average value of content transmission delay, respectively, for all vehicles under the current RSU range.

1) The Following Benchmark Algorithms are Used to Evaluate the Active Caching Scheme MAECC, (i.e., MAECCD Without the DRL Component), While Demonstrating the Superiority of the Proposed Predictive Content Popularity Algorithm: The MAECC scheme pre-caches the predicted popular content into the vehicle and RSU, and updates the cache of the RSU when the vehicle drives into the next RSU. The MAECCD scheme is based on the MAECC scheme, and updates the RSU and the vehicle's cache during the vehicle's driving, to further improve the network performance of the vehicle under the current RSU coverage.

- Random Cache(RC): randomly selects content from all databases to be cached into vehicles and RSUs.
- Thompson Sampling(TS) [38]: the user's historical request information is used to train the tuning of the content request probability parameter and finally the content cache with the highest probability is selected.
- Oracle [39]: select the content cache with the highest probability from the test set, so it possesses the most elevated cache hit rate among the active cache.
- ε-greedy: selects the most requested content cache with probability $1-\varepsilon$ and caches it randomly with probability ε.
- Federated averaging(FedAvg) [40]: FedAvg is a typical synchronous FL scheme where the RSU waits for all local models to be updated and then updates the global model using a weighted average.

Figs. 5 and 6 display the overall hit rate and content transmission delay for various active caching schemes with varying cache capacities. As the cache capacity increases, all schemes show an increase in hit rates and a decrease in content transmission delay. This is because more and more popular content is cached as the cache capacity increases. Random caching has the lowest hit rate and the highest content transmission delay, which is due to the fact that random caching does not take into account the effect of content popularity and just randomly selects content for caching. Compared to random caching, TS and ε-degree methods have shown to have better performance in selecting cached content by observing users' historical requests and using that information to determine what to cache. Compared to RC, TS, and ε -degree methods, the proposed MAECC scheme utilizes cached content that is based on a combination of factors, including the predicted content popularity, the user's historical requests, ratings, and interest preferences. As a result, it demonstrates superior performance in terms of hit rate and delay. Oracle has the best hit rate and content transmission delay because it knows the content of user vehicle requests in advance. In addition, the curves of the MAECC scheme are closer to those of Oracle also provide side evidence of the accuracy of our proposed content popularity prediction algorithm.

In Fig. 7, we compare the network performance in terms of transmission energy consumption and throughput for different cache capacities. From Fig. 7(a), it can be seen that as the cache capacity increases, the transmission energy consumption decreases. This is because the larger the capacity, the higher the probability of hitting files from the vehicle local and edge RSUs, thus reducing the energy consumption caused by retrieving content from remote BSs. Fig. 7(b) shows that the throughput of the algorithm increases as the cache capacity increases, because more files are transferred from the vehicle and edge RSU in the same time, and the network performance is better. Compared with other algorithms, the proposed algorithms have better network performance under different cache capacities, and the advantages are increasing with the increase of cache capacities.

The performance comparison for different vehicle densities is shown in Fig. 8, where the cache capacity is 200MB. As can

Authorized licensed use limited to: NORTHWESTERN POLYTECHNICAL UNIVERSITY. Downloaded on May 20,2026 at 02:37:52 UTC from IEEE Xplore. Restrictions apply.

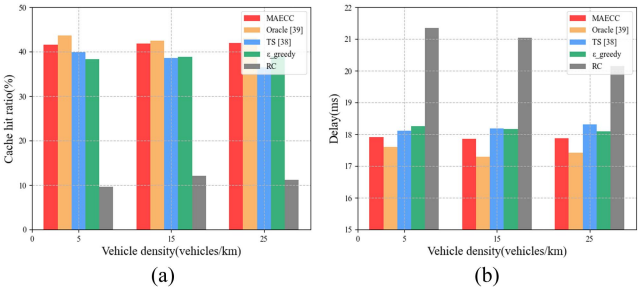


Fig. 8. Performance comparison of hit rate (a) and content transmission delay (b) under different vehicle densities. (a) Hit rate. (b) Content transmission delay.

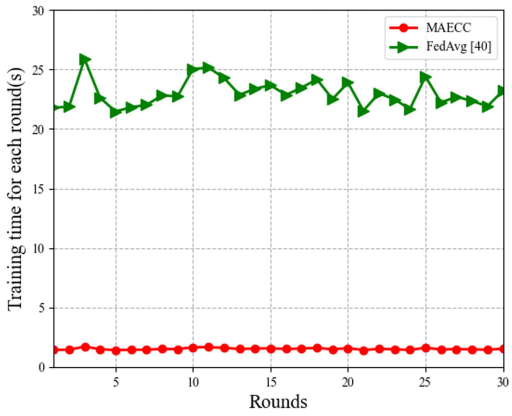


Fig. 9. Training time for asynchronous FL and FedAvg.

be seen in Fig. 8(a), the hit rate does not change much with the increase of vehicle density, because the increase of the number of vehicle requests is much larger than the increase of vehicle cache capacity. When the vehicle density continues to increase, the hit rate will be reduced due to the large number of requests. It can be found in Fig. 8(b) that with the increase of vehicle density, the delay has a downward trend. This is because with the increase of vehicles, there are more opportunities for vehicles to obtain content through V2V links, thus reducing the delay.

The Fig. 9 shows the comparison of the training time per round of the proposed asynchronous FL scheme with the synchronous FL scheme FedAvg when the vehicle density is 15 vehicles/km. It can be seen that the training time per round for the proposed asynchronous aggregation scheme is in the range of 1 to 2 s, while the training time per round for the FedAvg scheme is in the range of 21.5 to 25.5 s. Obviously, the proposed scheme significantly reduces the training time because of the use of asynchronous aggregation. Whereas the FedAvg scheme has a longer training time, this is because the scheme needs to wait for the local model updates of all vehicles before aggregation.

2) We Compare the Proposed Mobile-Aware Edge Cooperative Caching Scheme (MAECCD) With Different Baseline Algorithms: • Random Replace(RR): This algorithm randomly replaces cache contents when the cache reaches its maximum capacity.

• Least Recently Used(LRU): Once the capacity of the cache is full, the most recent content that has not been requested will be replaced.

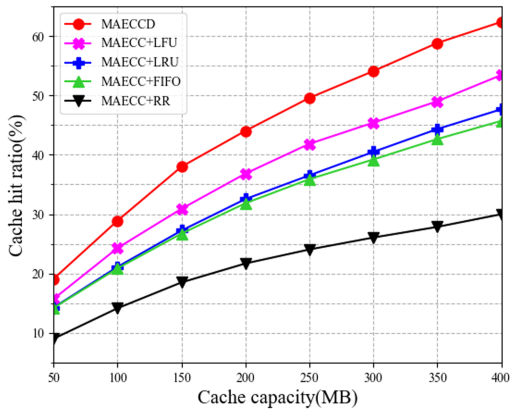


Fig. 10. Hit rate with different cache capacities.

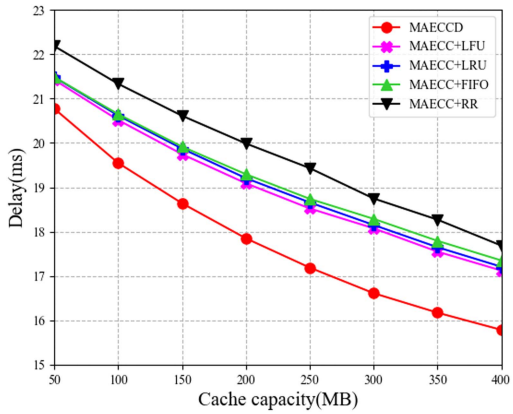


Fig. 11. Content transmission delay with different cache capacities.

• Least Frequently Used(LFU): Once the capacity of the cache is full, the least frequently requested content is replaced.

• First In First Out(FIFO): When the cache runs out of space, the first requested content is replaced.

In Fig. 10 we compare the hit rates of different update algorithms with different cache capacities using the active caching scheme MAECC, with a vehicle density of 15 vehicles/km. It is evident that the hit rate for all the update schemes increases as the cache capacity increases. This is because a larger the local cache can store more popular content, increasing the likelihood of a hit for a requested content by a vehicle. The LRU, LFU, FIFO, and Random Replacement algorithms are all based on the MAECC scheme. As expected, the Random Replacement algorithm has the lowest hit rate, as it randomly selects content to be replaced during the cache update process. On the other hand, the LRU, LFU, and FIFO algorithms take into account the requested content during the update process, resulting in improved performance compared to Random Replacement. The MAECCD scheme has the highest hit rate, as it considers the impact of the requested content and utilizes the DQN algorithm to update the RSU and vehicle caches. With a cache capacity of 400, our proposed scheme shows an average hit rate improvement of 32.4%, 16.7%, 14.7%, and 8.9% compared to the RR, FIFO, LRU, and LFU algorithms, respectively.

Authorized licensed use limited to: NORTHWESTERN POLYTECHNICAL UNIVERSITY. Downloaded on May 20,2026 at 02:37:52 UTC from IEEE Xplore. Restrictions apply.

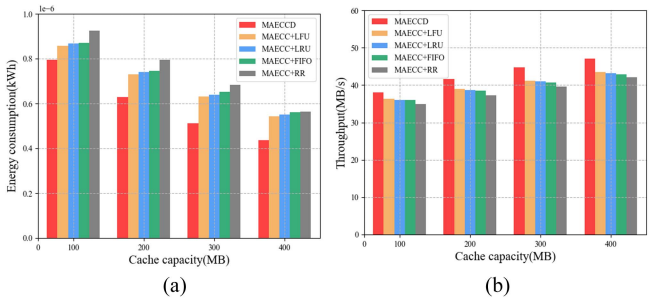


Fig. 12. Comparison of transmission energy consumption and throughput under different cache capacities. (a) Energy consumption. (b) Throughput.

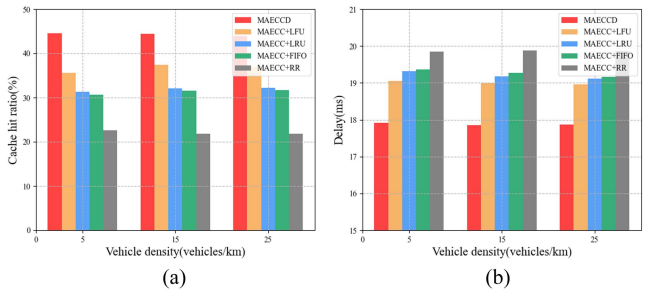


Fig. 13. Performance comparison of hit rate (a) and content transmission delay (b) under different vehicle densities. (a) Hit rate. (b) Content transmission delay.

In Fig. 11 we compare the hit rates of different update algorithms with different cache capacities using the active caching scheme MAECC, with a vehicle density of 15 vehicles/km. It is evident that the hit rate for all the update schemes increases as the cache capacity increases. This is because a larger the local cache can store more popular content, increasing the likelihood of a hit for a requested content by a vehicle. The LRU, LFU, FIFO, and Random Replacement algorithms are all based on the MAECC scheme. As expected, the Random Replacement algorithm has the lowest hit rate, as it randomly selects content to be replaced during the cache update process. On the other hand, the LRU, LFU, and FIFO algorithms take into account the requested content during the update process, resulting in improved performance compared to Random Replacement. The MAECCD scheme has the highest hit rate, as it considers the impact of the requested content and utilizes the DQN algorithm to update the RSU and vehicle caches. With a cache capacity of 400, our proposed scheme shows an average hit rate improvement of 32.4%, 16.7%, 14.7%, and 8.9% compared to the RR, FIFO, LRU, and LFU algorithms, respectively.

We compare the network performance of different update algorithms for different cache capacities by transmission energy consumption and throughput in Fig. 12. All the comparison schemes cache the popular content predicted by MAECC scheme, but it can be seen from the figure that MAECCD scheme has better performance than other algorithms by using DRL to update the cache. And with the increase of cache capacity, MAECCD scheme has greater advantages in terms of energy consumption and throughput compared with other schemes.

Fig. 13 show the request hit rate and content transmission delay of different algorithms under different vehicle densities when the cache capacity is 200MB. It is evident that the hit rate and delay of FIFO, LRU, LFU and random replacement algorithms do not significantly differ across different vehicle densities. This is because these algorithms are all based on the active caching scheme MAECC, which utilizes predicted content popularity based on user characteristics of each vehicle.With sufficient computational power, our proposed scheme accurately predicts the content popularity of the entire region and of individual vehicles at varying densities. In comparison to other schemes, our proposed MAECCD scheme utilizes vehicle user information to accurately predict content popularity and takes into account both content popularity and request information

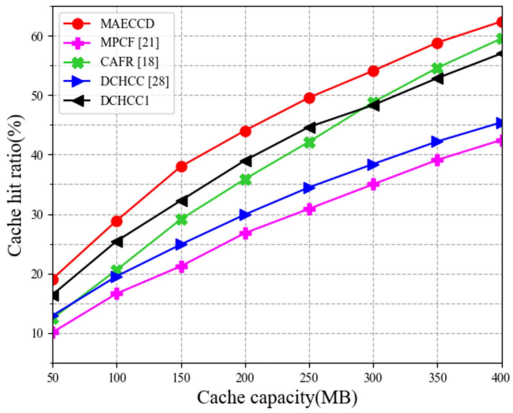


Fig. 14. Hit rate with different cache capacities.

when updating the cache. These factors provide an advantage at different vehicle densities.

3) Further Evaluate the Proposed MAECCD Scheme: • CAFR [18]: This approach utilizes an autoencoder to generate user ratings for unrated content, which are then used to predict the popularity of regional content. The DQN algorithm is also employed to update the local intra-RSU cache, taking into account cooperative caching between RSUs.

• MPCF [21]: This method utilizes a Context-aware Adversarial AutoEncoder to predict the highly dynamic content popularity, and dynamically updates the content cached by RSUs, but does not consider cooperative caching.

• DQN-based Cluster Head Content Caching(DCHCC) [28]: This approach clusters vehicles according to their mobility and preferences, and predicts content popularity by combining historical request frequency and preferences, and updates the vehicle cache content within the cluster using the DQN algorithm. However, it does not take into account cooperative caching.

• DCHCC1: This scheme builds upon the DCHCC approach by considering cooperative caching. It uses the next RSU in the direction of travel as a collaborative RSU to cache the popular content.

In Fig. 14 we compare the cache hit rates of varying cache schemes with different cache capacities, while maintaining a vehicle density of 15 vehicles/km. It is evident that the hit rates of all update schemes increase as the cache capacity increases. This is because a larger cache capacity allows for more content to be stored, increasing the likelihood of a vehicle's requested content

Authorized licensed use limited to: NORTHWESTERN POLYTECHNICAL UNIVERSITY. Downloaded on May 20,2026 at 02:37:52 UTC from IEEE Xplore. Restrictions apply.

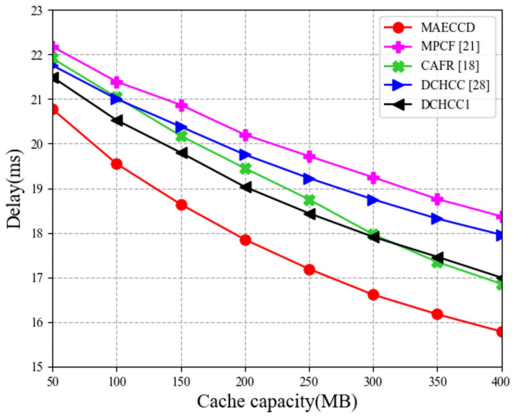


Fig. 15. Content transmission delay with different cache capacities.

being found in the cache. The MPCF and DCHCC schemes have lower hit rates compared to other algorithms because they do not take into account cooperative caching. However, the DCHCC scheme updates the local caches of the vehicles within a cluster through the cluster header, resulting in a higher local hit rate and better performance compared to MPCF. Both the CAFR and the DCHCC1 schemes consider cooperative caching, with DCHCC1 being more advantageous in cases of low RSU cache capacity, and CAFR being more advantageous in cases of large RSU cache capacity. This is due to the fact that when the RSU cache capacity is low, increasing the vehicle's local cache hit rate has a greater impact on the overall hit rate. As a result, the DCHCC1 scheme, which utilizes the DQN algorithm to update the vehicle's local cache, has a more significant improvement on the overall hit rate. As the capacity of the RSU cache increases, the CAFR scheme becomes more advantageous due to its ability to update the RSU cache content. Our proposed MAECCD algorithm, which incorporates cooperative caching and utilizes the DQN algorithm to update both the vehicle's local cache and the RSU cache contents, achieves the highest cache hit rate among the five algorithms compared in this paper.

Fig. 15 illustrates the content transmission delay of various cache schemes with different cache capacities at a vehicle density of 15 vehicles/km. It is evident that the delay decreases as the cache capacity increases. This is due to the fact that a larger cache capacity allows for more popular content to be cached, resulting in a higher cache hit rate, and ultimately reducing content transmission delay. Among all the algorithms, the MAECCD scheme has the lowest content transmission delay for different cache capacities. This can be attributed to the fact that the MAECCD scheme not only updates the cached content of the vehicle local and RSUs, but also takes into account cooperative caching. This leads to an improved overall cache hit rate, allowing vehicles to retrieve more requested content from their own cache, other vehicles in the cluster, and local RSUs. As a result, the content transmission delay is reduced.

In Fig. 16 we compare the transmission energy consumption and throughput of different algorithms. It can be seen from Fig. 16(a) that MPCF and DCHCC schemes need more energy consumption than other algorithms because they do not consider

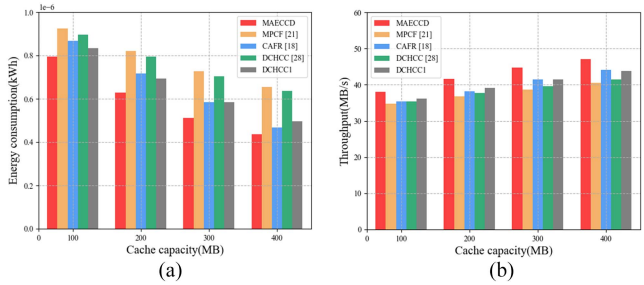


Fig. 16. Comparison of transmission energy consumption and throughput under different cache capacities. (a) Energy consumption. (b) Throughput.

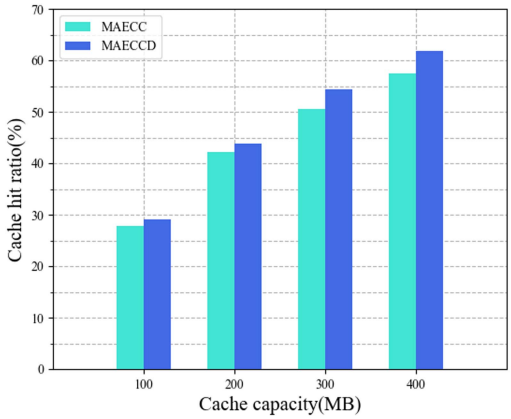


Fig. 17. Hit rate with different cache capacities.

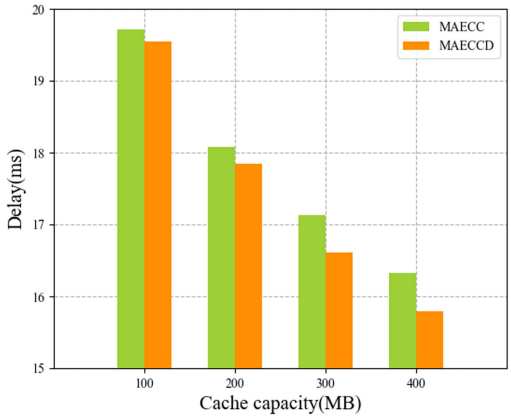


Fig. 18. Content transmission delay with different cache capacities.

cooperative caching. This is because the energy consumption required to hit the file through cooperative caching is far less than the energy consumption required to obtain the file from the base station. MAECCD has the best performance among the schemes considering cooperative caching. It can be seen from Fig. 16(b) that the proposed algorithm has the best performance compared with other algorithms in terms of throughput.

Figs. 17 and 18 illustrate the comparison between the MAECCD scheme with different cache capacities and the MAECC scheme. As the cache capacity increases, both schemes are able to cache more popular content, resulting in a gradual

Authorized licensed use limited to: NORTHWESTERN POLYTECHNICAL UNIVERSITY. Downloaded on May 20,2026 at 02:37:52 UTC from IEEE Xplore. Restrictions apply.

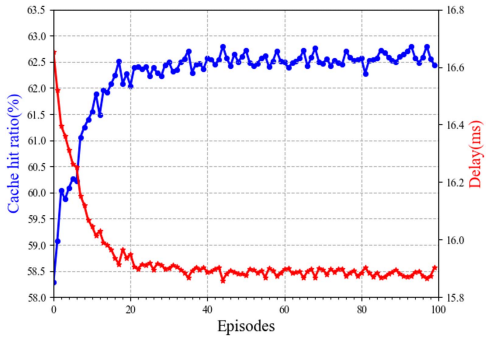


Fig. 19. Cache hit rate and delay for each episodes in DRL training.

increase in the hit rate and a decrease in content transmission delay. After the MAECCD scheme actively caches popular content, both the vehicle and RSUs will continuously update the cached content based on user requests and content popularity. This results in an improved cache hit rate and reduced content transmission delay. Additionally, as the cache capacity increases, the benefits of the MAECCD scheme become increasingly apparent. For example, with a cache capacity of 400, the average hit rate of the MAECCD scheme is further improved by 4.3% compared to the MAECC scheme.

Fig. 19 depicts the correlation between cache hit rate and content transmission delay for various training rounds, with a fixed RSU cache capacity of 400. As the number of training rounds increases, the hit rate of vehicle requests gradually improves and the content transfer delay gradually decreases. This is due to the fact that during this period, the local caches of the vehicles and RSUs are further updated by continuously learning from the training. There is a gradual convergence around 20 rounds to update the cache to the optimal state.

VII. Conclusion

In this paper, we propose a Mobile-Aware Edge cooperative Caching scheme (MAECCD) that combines FL and DRL to address the issue of excessive response delay caused by increased data traffic in IoV. Our approach utilizes an autoencoder to capture higher-order features of vehicle user data and predict user preferences for unrated content and different themes. This allows us to obtain the overall content popularity under the vehicle's local and RSU coverage. To protect user privacy, the training process is distributed to individual vehicles using FL. Additionally, our MAECCD scheme takes advantage of the high mobility and flexibility of vehicles by caching the predicted popularity as the vehicle moves to the next RSU. This ensures the validity of the content popularity and improves the matching between vehicles. To update the cache contents of both the RSU and the vehicle, we utilize the Dueling DQN algorithm in DRL. Furthermore, we consider the collaboration problem among the nodes at the edge to increase the cache utilization of the edge devices and further decrease the content delivery delay. The experimental simulation results demonstrate that MAECCD outperforms other algorithms in terms of hit rate and content transmission delay reduction.

References

[1] J. Zhang and K. B. Letaief, "Mobile edge intelligence and computing for the Internet of Vehicles," *Proc. IEEE*, vol. 108, no. 2, pp. 246–261, Feb. 2020.
[2] S. Wang, X. Zhang, Y. Zhang, L. Wang, J. Yang, and W. Wang, "A survey on mobile edge networks: Convergence of computing, caching and communications," *IEEE Access*, vol. 5, pp. 6757–6779, 2017.
[3] H. Yu, M. H. Cheung, G. Iosifidis, L. Gao, L. Tassiulas, and J. Huang, "Mobile data offloading for green wireless networks," *IEEE Wireless Commun.*, vol. 24, no. 4, pp. 31–37, Aug. 2017.
[4] N. Abbas, Y. Zhang, A. Taherkordi, and T. Skeie, "Mobile edge computing: A survey," *IEEE Internet Things J.*, vol. 5, no. 1, pp. 450–465, Feb. 2018.
[5] T. Taleb, S. Dutta, A. Ksentini, M. Iqbal, and H. Flinck, "Mobile edge computing potential in making cities smarter," *IEEE Commun. Mag.*, vol. 55, no. 3, pp. 38–43, Mar. 2017.
[6] Y. Sun, Z. Wu, D. Shi, and X. Hu, "Task offloading method of Internet of Vehicles based on cloud-edge computing," in *Proc. 2022 IEEE Int. Conf. Serv. Comput.*, 2022, pp. 315–320.
[7] T. Taleb, K. Samdanis, B. Mada, H. Flinck, S. Dutta, and D. Sabella, "On multi-access edge computing: A survey of the emerging 5G network edge cloud architecture and orchestration," *IEEE Commun. Surveys Tuts.*, vol. 19, no. 3, pp. 1657–1681, Thirdquarter 2017.
[8] H. Peng, L. Liang, X. Shen, and G. Y. Li, "Vehicular communications: A network layer perspective," *IEEE Trans. Veh. Technol.*, vol. 68, no. 2, pp. 1064–1078, Feb. 2019.
[9] Z. Zhang, C.-H. Lung, M. St-Hilaire, and I. Lambadaris, "Smart proactive caching: Empower the video delivery for autonomous vehicles in ICN-based networks," *IEEE Trans. Veh. Technol.*, vol. 69, no. 7, pp. 7955–7965, Jul. 2020.
[10] J. Erman, A. Gerber, M. Hajiaghayi, D. Pei, S. Sen, and O. Spatscheck, "To cache or not to cache: The 3G case," *IEEE Internet Comput.*, vol. 15, no. 2, pp. 27–34, Mar./Apr. 2011.
[11] R. Fares, B. Romoser, Z. Zong, M. Nijim, and X. Qin, "Performance evaluation of traditional caching policies on a large system with petabytes of data," in *Proc. 2012 IEEE 7th Int. Conf. Netw., Architecture, Storage*, 2012, pp. 227–234.
[12] M. Yan, C. A. Chan, W. Li, L. Lei, A. F. Gygax, and I. Chih-Lin, "Assessing the energy consumption of proactive mobile edge caching in wireless networks," *IEEE Access*, vol. 7, pp. 104394–104404, 2019.
[13] X. Wang, Y. Han, C. Wang, Q. Zhao, X. Chen, and M. Chen, "In-edge AI: Intelligentizing mobile edge computing, caching and communication by federated learning," *IEEE Netw.*, vol. 33, no. 5, pp. 156–165, Sep./Oct. 2019.
[14] Z. Yu et al., "Federated learning based proactive content caching in edge computing," in *Proc. 2018 IEEE Glob. Commun. Conf.*, 2018, pp. 1–6.
[15] W. Huang, T. Song, Y. Yang, and Y. Zhang, "Cluster-based cooperative caching with mobility prediction in vehicular named data networking," *IEEE Access*, vol. 7, pp. 23442–23458, 2019.
[16] Y. M. Saputra, D. T. Hoang, D. N. Nguyen, and E. Dutkiewicz, "A novel mobile edge network architecture with joint caching-delivering and horizontal cooperation," *IEEE Trans. Mobile Comput.*, vol. 20, no. 1, pp. 19–31, Jan. 2021.
[17] Y. Liu and B. Mao, "On a novel content edge caching approach based on multi-agent federated reinforcement learning in Internet of Vehicles," in *Proc. 32nd Wireless Opt. Commun. Conf.*, 2023, pp. 1–5.
[18] Q. Wu, Y. Zhao, Q. Fan, P. Fan, J. Wang, and C. Zhang, "Mobility-aware cooperative caching in vehicular edge computing based on asynchronous federated and deep reinforcement learning," *IEEE J. Sel. Topics Signal Process.*, vol. 17, no. 1, pp. 66–81, Jan. 2023.
[19] R. Wang, Z. Kan, Y. Cui, D. Wu, and Y. Zhen, "Cooperative caching strategy with content request prediction in Internet of Vehicles," *IEEE Internet Things J.*, vol. 8, no. 11, pp. 8964–8975, Jun. 2021.
[20] Z. Yu, J. Hu, G. Min, H. Xu, and J. Mills, "Proactive content caching for Internet-of-Vehicles based on peer-to-peer federated learning," in *Proc. 2020 IEEE 26th Int. Conf. Parallel Distrib. Syst.*, 2020, pp. 601–608.
[21] Z. Yu, J. Hu, G. Min, Z. Zhao, W. Miao, and M. S. Hossain, "Mobility-aware proactive edge caching for connected vehicles using federated learning," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 8, pp. 5341–5351, Aug. 2021.
[22] W. Wang, Y. Zhao, Q. Wu, Q. Fan, C. Zhang, and Z. Li, "Asynchronous federated learning based mobility-aware caching in vehicular edge computing," in *Proc. 2022 14th Int. Conf. Wireless Commun. Signal Process.*, 2022, pp. 1–5.

Authorized licensed use limited to: NORTHWESTERN POLYTECHNICAL UNIVERSITY. Downloaded on May 20,2026 at 02:37:52 UTC from IEEE Xplore. Restrictions apply.

[23] Y. Dai, D. Xu, Y. Lu, S. Maharjan, and Y. Zhang, "Deep reinforcement learning for edge caching and content delivery in Internet of Vehicles," in *Proc. 2019 IEEE/CIC Int. Conf. Commun. China*, 2019, pp. 134–139.
[24] K. Jiang, H. Zhou, D. Zeng, and J. Wu, "Multi-agent reinforcement learning for cooperative edge caching in Internet of Vehicles," in *Proc. 2020 IEEE 17th Int. Conf. Mobile Ad Hoc Sensor Syst.*, 2020, pp. 455–463.
[25] D. Zhang, W. Wang, J. Zhang, T. Zhang, J. Du, and C. Yang, "Novel edge caching approach based on multi-agent deep reinforcement learning for Internet of Vehicles," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 8, pp. 8324–8338, Aug. 2023.
[26] C. Li, Y. Zhang, and Y. Luo, "A federated learning-based edge caching approach for mobile edge computing-enabled intelligent connected vehicles," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 3, pp. 3360–3369, Mar. 2023.
[27] Y. Lu, X. Huang, K. Zhang, S. Maharjan, and Y. Zhang, "Blockchain empowered asynchronous federated learning for secure data sharing in Internet of Vehicles," *IEEE Trans. Veh. Technol.*, vol. 69, no. 4, pp. 4298–4311, Apr. 2020.
[28] Y. Wang, K. Zheng, W. Ye, and Y. Tang, "Popularity-aware caching for vehicle clusters with federated deep reinforcement learning," *IEEE Commun. Lett.*, vol. 27, no. 6, pp. 1644–1648, Jun. 2023.
[29] G. Yu, Y. He, J. Wu, Z. Chen, and J. Pan, "Mobility-aware proactive edge caching for large files in the Internet of Vehicles," *IEEE Internet Things J.*, vol. 10, no. 13, pp. 11293–11305, Jul. 2023.
[30] 3GPP, "Study on lte-based v2x services," 3GPP, Sophia Antipolis, France, Tech. Rep. TR 36.885, 2016.
[31] J. Chen, H. Wu, P. Yang, F. Lyu, and X. Shen, "Cooperative edge caching with location-based and popular contents for vehicular networks," *IEEE Trans. Veh. Technol.*, vol. 69, no. 9, pp. 10291–10305, Sep. 2020.
[32] P. Wang and H. Ye, "A personalized recommendation algorithm combining slope one scheme and user based collaborative filtering," in *Proc. 2009 Int. Conf. Ind. Inf. Syst.*, 2009, pp. 152–154.
[33] T. Zhang, H. Fan, J. Loo, and D. Liu, "User preference aware caching deployment for device-to-device caching networks," *IEEE Syst. J.*, vol. 13, no. 1, pp. 226–237, Mar. 2019.
[34] Q. Wu, P. Huang, Q. Zheng, and H. Song, "A segmented d2d caching strategy based on user preferences and overall popularity," in *Proc. 2023 8th Int. Conf. Comput. Commun. Syst.*, 2023, pp. 239–246.
[35] C. Fahy, S. Yang, and M. Gongora, "Ant colony stream clustering: A fast density clustering algorithm for dynamic data streams," *IEEE Trans. Cybern.*, vol. 49, no. 6, pp. 2215–2228, Jun. 2019.
[36] Z. Wang, T. Schaul, M. Hessel, H. van Hasselt, M. Lanctot, and N. de Freitas, "Dueling network architectures for deep reinforcement learning," in *Proc. 33rd Int. Conf. Int. Conf. Mach. Learn.*, 2016, pp. 1995–2003.
[37] F. M. Harper and J. A. Konstan, "The movielens datasets: History and context," *ACM Trans. Interact. Intell. Syst.*, vol. 5, no. 4, pp. 1–19, Dec. 2015, doi: 10.1145/2827872.
[38] L. Cui et al., "CREAT: Blockchain-assisted compression algorithm of federated learning for content caching in edge computing," *IEEE Internet Things J.*, vol. 9, no. 16, pp. 14151–14161, Aug. 2022.
[39] X. Wang, C. Wang, X. Li, V. C. M. Leung, and T. Taleb, "Federated deep reinforcement learning for Internet of Things with decentralized cooperative edge caching," *IEEE Internet Things J.*, vol. 7, no. 10, pp. 9441–9455, Oct. 2020.
[40] J. Konečný, H. B. McMahan, D. Ramage, and P. Richtárik, "Federated optimization: Distributed machine learning for on-device intelligence," 2016, *arXiv:1610.02527*.



Weidi Tian received the B.S. degree in electronic information science and technology from Shanxi University, Taiyuan, China, in 2022. He is currently working toward the M.S. degree in communication engineering with South China Normal University, Guangzhou, China. His research focuses on IoV, mobile edge computing, and deep reinforcement learning.



Yujian Chen is currently working toward the M.S. degree with the School of Electronic Science and Engineering (School of Microelectronics), South China Normal University, Guangzhou, China. His research interests include vehicular networks and edge caching.



Feng Ke received the Ph.D. degree from the School of Electronic and Information Engineering, South China University of Technology (SCUT), Guangzhou, China. From 2013 to 2014, he was a Visiting Scholar with the Department of Electronic Engineering, Missouri University of Science and Technology, Rolla, MO, USA. He is currently the Vice Director of the Intelligent Communication Network and Computer Engineering Technology Engineering Center, SCUT and Chief Scientist of Guangdong Chinese Network Electrical Engineering Research Institute. He is also an Expert Advisor with Guangzhou Science and Technology Bureau and Guangdong Science and Technology Department. He has authored or coauthored more than 60 research papers in authoritative journals and conferences, including IEEE TRANSACTIONS ON COMMUNICATIONS, IEEE TRANSACTIONS ON VEHICULAR TECHNOLOGY, and GLOBECOM. His research interests include communications and information theory with special emphasis on wireless communications and signal processing, wireless body area networks, dynamic network architecture.



Hui Song received the Ph.D. degree in communication and information system from the South China University of Technology (SCUT), Guangzhou, China, in 2005. She is currently an Associate Professor with the School of Electronic Science and Engineering (School of Microelectronics), and Physics and Telecommunication, South China Normal University (SCNU), Guangzhou. Her research interests include strategy design and optimization in D2D network and crowd-sensing network.

Authorized licensed use limited to: NORTHWESTERN POLYTECHNICAL UNIVERSITY. Downloaded on May 20,2026 at 02:37:52 UTC from IEEE Xplore. Restrictions apply.